

University of Macau

Faculty of Science and Technology



澳門大學
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU

**Product Recommendation System Based on Large Language Model
and User-Item collaboration**

by

CUI JIANFU , Student No: DC127910

Graduation Project Report submitted in partial fulfillment
of the requirements of the Degree of
Bachelor of Science in Computer Science

Project Supervisor

Prof. Zhang Liming

20 May 2025

DECLARATION

I sincerely declare that:

1. I and my teammates are the sole authors of this report,
2. All the information contained in this report is certain and correct to the best of my knowledge,
3. I declare that the thesis here submitted is original except for the source materials explicitly acknowledged and that this thesis or parts of this thesis have not been previously submitted for the same degree or for a different degree, and
4. I also acknowledge that I am aware of the Rules on Handling Student Academic Dishonesty and the Regulations of the Student Discipline of the University of Macau.
5. I acknowledge that AI (<https://chat.deepseek.com/>) helped me interpret error messages in tasks and helped me translate text and improve my language expression in reports.



Signature :

Name : Chris CUI JIANFU

Student No. : DC127910

Date : 20 May 2025

ACKNOWLEDGEMENTS

I express my deepest gratitude to the University of Macau and the Faculty of Science and Technology for providing me with the resources and opportunity to undertake this graduation project. The academic environment and rigorous training I received have laid a solid foundation for my research capabilities and practical skills.

I am particularly indebted to my supervisor, Prof. ZHANG LIMING, for her invaluable guidance, persistent encouragement, and insightful feedback throughout the entire project. Her expertise in the field and thoughtful advice were instrumental in shaping my research direction, refining the methodology, and ultimately achieving meaningful outcomes.

I also wish to thank my project teammate, Jacky, for the many ideas and details that emerged from our frequent communication and sharing. I am grateful to the professors of the relevant courses for their generous teaching, as well as to my fellow students for their academic discussions and constructive suggestions. Their input and shared resources have been tremendously helpful to my work.

Finally, I am truly thankful to my family and friends for their understanding and support. They provided the material backing and emotional strength that enabled me to remain focused and dedicated to the project.

ABSTRACT

This report presents our design and ongoing development of an intelligent product search and recommendation platform that leverages multi-modal data processing, large language models (LLMs), and User_Item collaboration to enhance retrieval precision and recommendation accuracy. By integrating natural language processing with computer vision and a user-item collaboration, our system aims to offer both highly relevant search results and personalized product suggestions. We have established the core architecture, implemented basic filter and search functionality, and tested a multi-modal similarity-based recommendation method on sample data. In parallel, the platform employs embedding techniques (OpenCLIP) to encode product images and text descriptions into a shared vector space, followed by Faiss indexing for scalable similarity searches. Preliminary evaluations on real or simulated datasets demonstrate notable improvements in recall, precision, and overall retrieval relevance when compared to traditional keyword-based methods. We will also highlights key technical challenges — such as optimizing multi-modal fusion and efficiently managing large-scale data — and outlines planned enhancements to further elevate system performance and user experience.

TABLE OF CONTENTS

CHAPTER 1.INTRODUCTION	8
1.1 Background	8
1.2 Objective	8
1.3 Outline	9
CHAPTER 2.LITERATURE SURVEY/RELATED WORK	10
2.1 Evolution of E-commerce Recommendation	10
2.2 Similarity Model	14
CHAPTER 3.PROJECT EXECUTION SCHEDULE	16
CHAPTER 4.METHOD	18
4.1 Data Preprocessing	18
4.1.1 Data Correction and Statistical Analysis	18
4.1.2 Dataset Reclassification	18
4.2 Model Training and Fine-Tuning	20
4.2.1 Retrieval Model	20
4.2.2 Multimodal Similarity Model	21
CHAPTER 5. IMPLEMENTATION	25
5.1 Code Structure Overview	25
5.2 Model Training and Finetuning	27
5.2.1 Retrieval Model	27
5.2.2 Multimodal Similarity Model	28
5.3 Recommendation	29
5.3.1 Similarity-Based Rec	29
5.3.2 User-Item Interaction Rec	29
5.4 UI and User Interaction	31
CHAPTER 6. RESULT(TESTING)	34
6.1 Terminal Output	34
6.2 Samples	35
6.3 System	35
CHAPTER 7. DISCUSSION	40

CHAPTER 8. ETHIC CONDUCT	41
CHAPTER 9. CONCLUSION	42
CHAPTER 10. REFERENCE	43

LIST OF FIGURES

FIGURE 1: GRAPHIC EMBEDDING.....	10
FIGURE 2: THE ARCHITECTURE OF TAOBAO SEARCHENGINE.....	11
FIGURE 3: ARCHITECTURAL DIAGRAM FOR THE RECOMMENDER SYSTEM.....	12
FIGURE 4: RELATED COMMON STRATEGIES.....	13
FIGURE 5: UNDERSTANDING WITH BERT.....	14
FIGURE 6: GANTT CHART.....	17
FIGURE 7: OPENCLIP ARCHITECTURE DIAGRAM.....	19
FIGURE 8: FLAN-T5 ARCHITECTURE DIAGRAM.....	19
FIGURE 9: RESNET50 ARCHITECTURE DIAGRAM.....	22
FIGURE 10: BERT ARCHITECTURE DIAGRAM.....	22
FIGURE 11: MODEL TRAINING FITTING CURVE.....	34
FIGURE 12: UI(GENERAL BROWSING).....	36
FIGURE 13: UI(SEARCH).....	37
FIGURE 14: UI (PRODUCT DETAILS).....	37
FIGURE 15: UI (SIMILARITY RECOMMENDATION).....	38
FIGURE 16: UI (USER-ITEM RECOMMENDATION).....	39

CHAPTER 1. INTRODUCTION

1.1 Background

In recent years, the booming development of e-commerce has brought consumers a new shopping experience, and online shopping has become an indispensable part of people's daily lives. In this context, the performance of product search and recommendation systems directly affects users' shopping satisfaction. Existing recommendation technologies mainly rely on keyword matching or user behavior analysis. Although they can meet basic needs, there are still many pain points in practical applications. For example, the inaccurate product title and description information, the incomplete user portrait, and the insufficient new user behavior data may lead to deviations between the recommendation results and the user's real preferences.

It is worth noting that the latest progress in the field of artificial intelligence has brought new possibilities for solving these problems, especially multimodal learning models, which show excellent image and text understanding capabilities. At the same time, user behavior analysis technology has also made significant progress. These technological breakthroughs have created favorable conditions for building more accurate and personalized intelligent recommendation systems. At present, how to effectively integrate multi-source data and combine semantic understanding and behavior analysis to optimize recommendation effects is becoming a key exploration direction for e-commerce platforms.

1.2 Objective

Although the search and recommendation functions of current e-commerce platforms are already quite popular, there are still some pain points in actual use. For example, when users search for products, they often encounter situations where the system cannot accurately understand the real needs. They clearly enter specific keywords, but return a large number of irrelevant results. Or when browsing recommended products, they find that the system repeatedly pushes products that have been purchased or are not interested in at all, and the degree of personalization needs to be improved. The root cause of these problems is that traditional recommendation systems do not have a deep understanding of product information, and the analysis of user behavior is also relatively scarce.

In response to these practical problems, we plan to implement a richer and more accurate search recommendation system. This system will achieve more accurate product matching by combining the latest multimodal understanding technology and user behavior analysis. Specifically, the system can not only read the product description text, but also understand the visual information contained in the product image, so as to grasp the product characteristics more comprehensively. In addition to the mining and matching of graphic information, the system will also analyze the user's purchase preference history and establish a more detailed user portrait, striving to "understand what you want and push what you need".

At the same time, in order to make this system easy to use, we also focus on optimizing the user experience. The system interface will be designed to be simple and intuitive, search results will be intelligently sorted according to multiple criteria, and recommended products will be dynamically adjusted based on user feedback. We believe that this will not only help users find their favorite products faster, but also bring better conversion results to merchants in possible practical applications.

1.3 Outline

In terms of data preparation, we use the public data set of Amazon UK in 2023 as the basis. The focus of this stage is to deeply clean and reclassify the original data, eliminate invalid information, correct erroneous data, and normalize product features. Through systematic inspection and analysis, we will ensure that the data used in the end is both accurate and reliable, and can fully reflect the real e-commerce environment.

In the core technology research and development part, we advance in two steps: the first step is to build an efficient retrieval system based on OpenCLIP and Faiss. This combination can quickly match users' text queries with product images, greatly improving search efficiency; the second step is to develop a more sophisticated multimodal matching model. By combining the image understanding ability of ResNet50 and the text analysis advantages of BERT, the system can capture the deep connection between product images and texts, thereby giving more accurate recommendation results.

In order to enhance the practicality of the system, we specially designed a user behavior analysis module. With the help of VADER sentiment analysis tools and spaCy natural language processing technology, the system can automatically extract key information from user comments and generate intuitive keyword cloud maps. These analysis results can not only help users quickly grasp the characteristics of products, but also accumulate valuable user preference data for the system to achieve intelligent recommendations across categories.

In terms of user experience optimization, we developed a simple and easy-to-use desktop application. This interface integrates core functions such as search, filtering, and recommendation. The operation process is carefully designed to ensure that users can easily find the products they need.

The entire project has always been practical-oriented to ensure that the final results can truly meet the actual needs of e-commerce platforms and consumers.

CHAPTER 2. Literature Survey/Related Work

2.1 Evolution of E-commerce Recommendation

In the mid-1990s, Amazon and eBay pioneered the combination of online product display and search engines, using ASIN-keyword inverted index to manage product data. In the early days of e-commerce, the technical focus of the platform was on the most basic page response speed and payment security. Product display was still static HTML pages, and inventory management mainly relied on traditional databases. As for the personalized recommendations and intelligent searches that are now commonplace, they were still in their infancy at the time.

After entering the Web 2.0 era, with the rapid expansion of the e-commerce industry, the pressure faced by the platform has increased day by day. Massive commodity data and uneven classification standards have made traditional indexing technology stretched. Although recommendation algorithms such as collaborative filtering were already quite mature at the time, problems such as data sparsity and cold start have always plagued major platforms. It was not until the BGE/EGES algorithm proposed by Alibaba at the KDD conference truly alleviated these problems on an industrial scale. This technology provides more accurate recommendations for Taobao's "Guess You Like" function by mapping products and their attributes into vector space. [1]

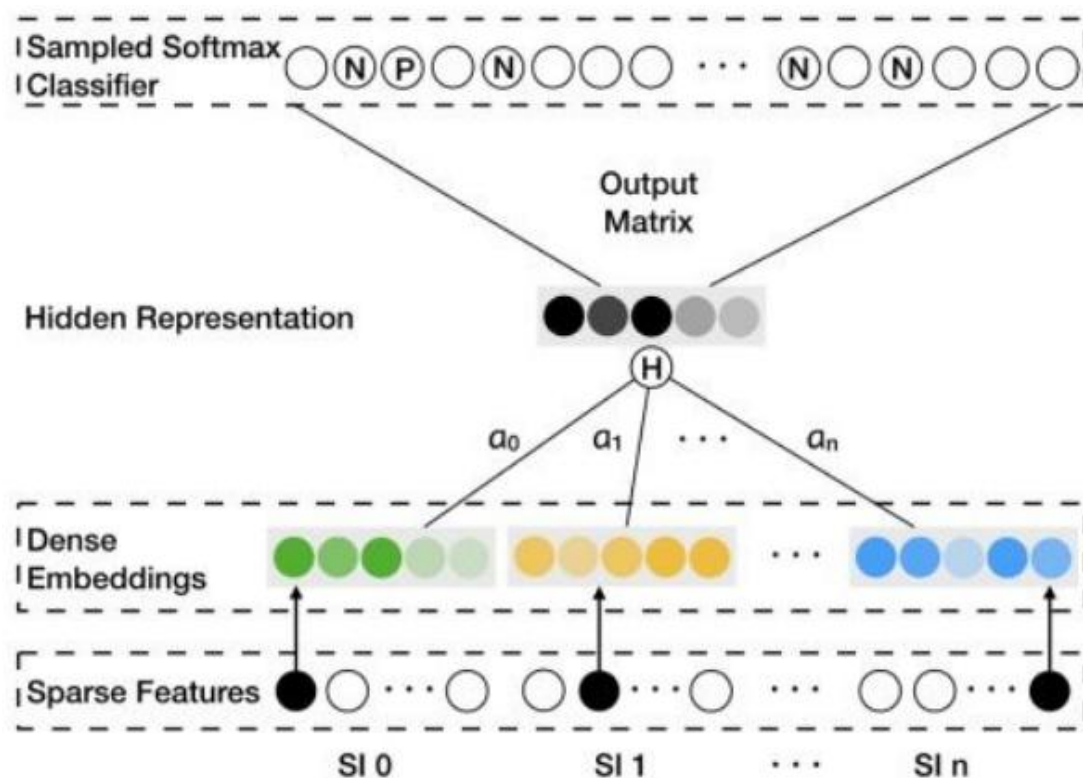


Figure 1. Graphic Embedding

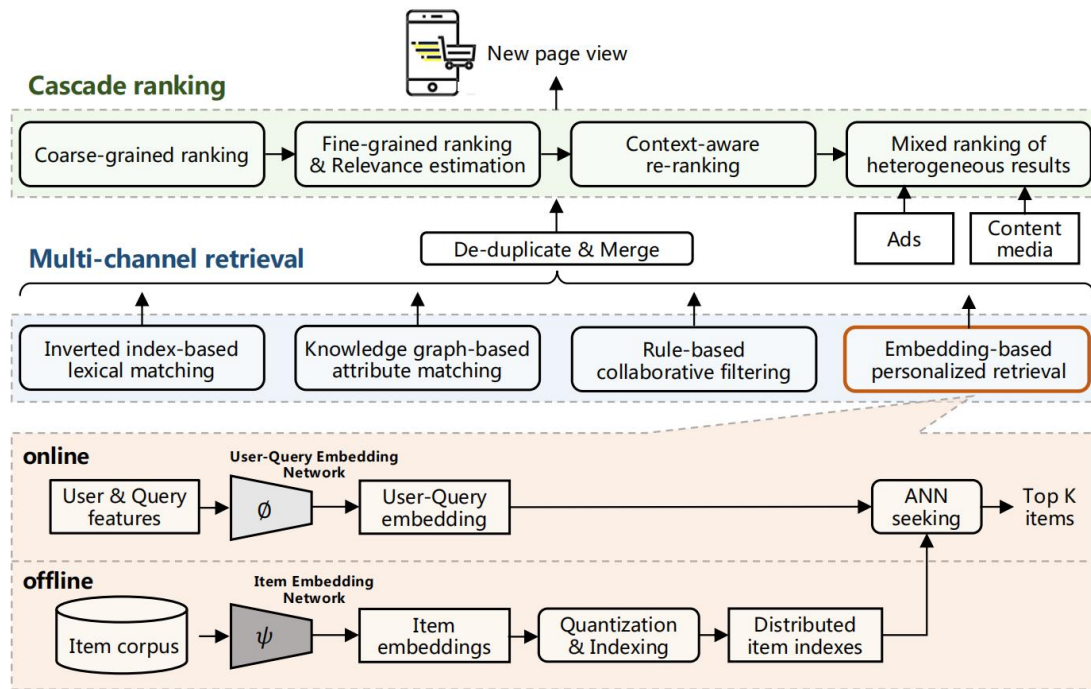


Figure 2 : The architecture of Taobao search engine with our embedding-based retrieval method

Around 2020, with the explosion of short videos and live streaming, the form of product display has undergone a qualitative leap, upgrading from simple text descriptions to a multi-dimensional presentation that integrates images, text, videos, and user reviews. This change makes traditional keyword searches seem inadequate, and major e-commerce platforms have to speed up the application of vector retrieval technology.

Take eBay as an example. At that time, the number of active products on the platform had exceeded 1.5 billion, and the speed of new products being put on the shelves was calculated in seconds. Faced with the problem of reduced search accuracy caused by multilingual titles and sellers' custom classifications, the eBay R&D team launched an Embedding-based retrieval system. This system uses a dual-tower model to uniformly encode product titles, main images, and user click behaviors into vectors, and then performs real-time similarity calculations through GPU clusters. The actual application effect is surprising: the update speed of recommended content has increased from hours to minutes, and the homepage click-through rate and user retention rate have increased by 6% and 3% respectively. [2]

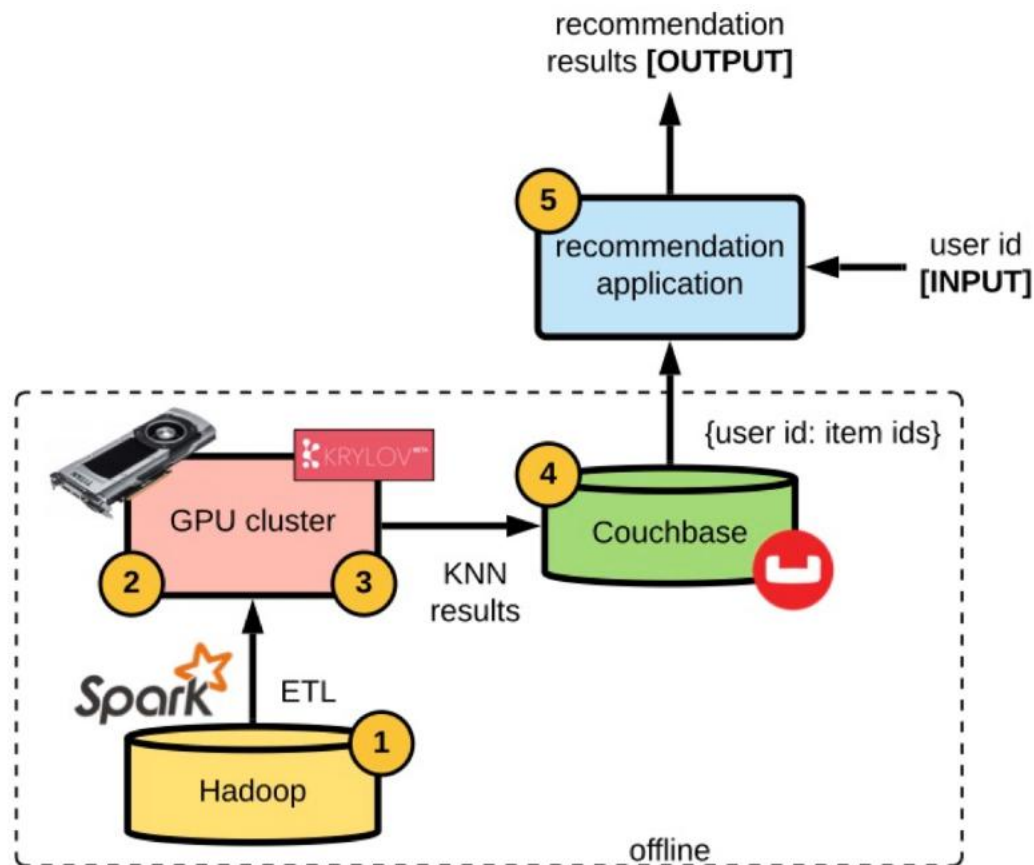


Figure 3: Architectural diagram for the recommender system.

JD.com also handed in its answer sheet in 2021. The "from semantic retrieval to pairing sorting" framework they developed first replaces traditional keyword matching with a deep semantic model to achieve millisecond-level product recall, and then uses a multi-objective learning algorithm to comprehensively consider factors such as conversion rate, merchant exposure, and logistics timeliness for accurate sorting. With the help of Flink to process user behavior data in real time, hot-selling new products in the live broadcast room can be included in the recommendation list within a few minutes. Actual test data shows that this system has increased the conversion rate of "buy while watching" by more than 4%, and the user browsing depth has also increased by 5-6 percentage points. [3]

However, with the rapid development of LLM and cross-modal large models, the industry has a more systematic way to explore the fine-grained semantics contained in product titles, descriptions, main images, and even short videos. The integration of "image-text-behavior" has gradually become the technical axis of e-commerce intelligence. Among the many relatively easy-to-implement approaches, the emergence of open source projects such as CLIP provides support for this. It pre-trains a large number of image-text pairs, maps product images and texts to the same vector space, and enables natural language queries to directly match product images or titles.

Projects such as OpenCLIP follow CLIP's contrastive learning framework: on hundreds of millions of image-text pairs, with the goal of "image features·text features → same dimensional vectors", minimize the "positive pair" cosine distance and maximize the "negative pair" distance, so that the visual and language channels are aligned in the same semantic space. In this way, even if natural language queries have multiple overlapping query qualifiers, only one vectorization is needed to perform approximate nearest neighbor retrieval in the product gallery, eliminating the need for multi-level keyword and rule rewriting. [4] [5]

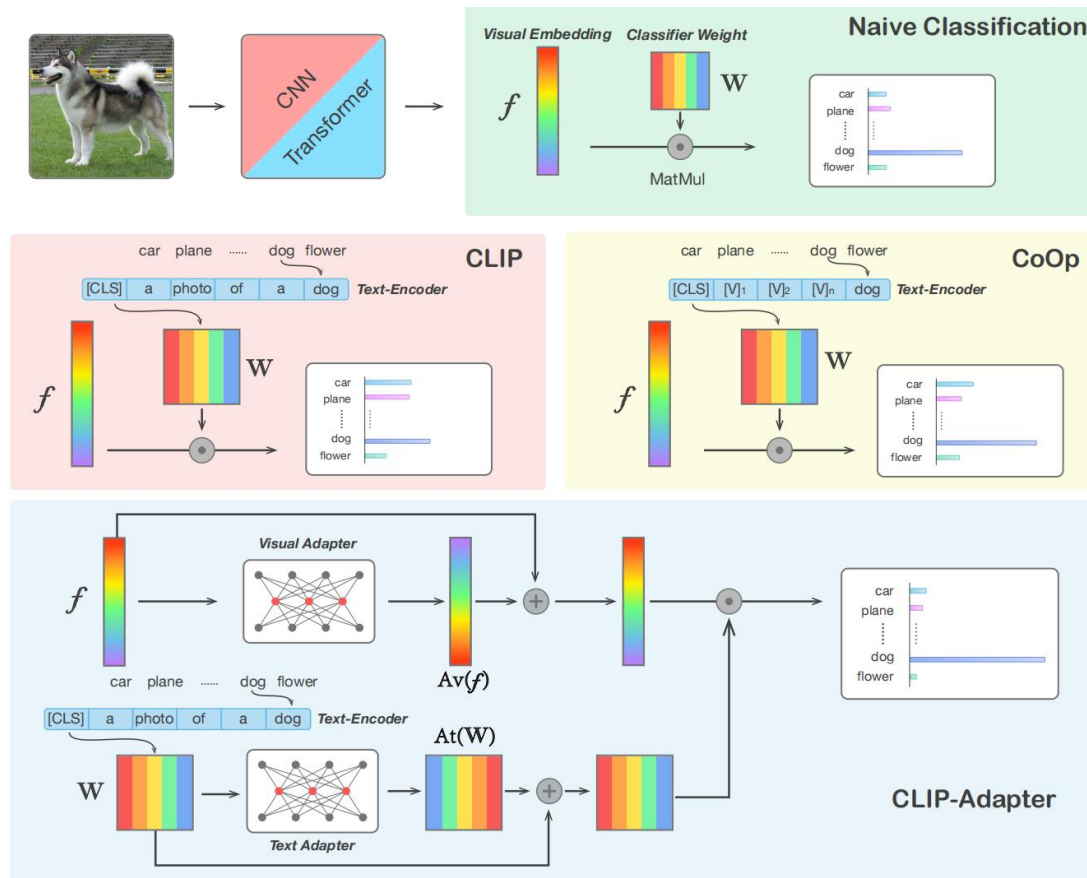


figure 4. Related common strategies

However, LLM reasoning is expensive and very sensitive to latency, so the industry generally adopts a "two-stage retrieval" process: first, in the rough recall stage, collaborative filtering (CF) or Faiss vector neighbors are used to quickly extract $N \approx 200-1,000$ candidates; then in the fine ranking stage, LLM only performs deep interactive evaluation (cross-encoder/listwise prompt) on this batch of candidates, re-ranks them with more detailed semantic understanding, and finally retains the top $k \approx 20-50$. This not only maintains millisecond-level response, but also makes the LLM reasoning budget "spend wisely".

In 2019, Google Search deployed the BERT ranking model globally, becoming a benchmark case for the implementation of semantic search technology. Its technical

route is quite representative: first, hundreds of candidate web pages are quickly recalled through the traditional inverted index, and then the BERT model is used to perform deep semantic matching between search keywords and web page content, and finally the results are rearranged. According to Google's official disclosure, this technology quickly covered almost all English search requests, especially showing significant advantages in understanding complex long sentences and daily spoken expressions. This case not only established the search technology paradigm of "casting a wide net first and then accurately screening", but also provided engineering practices that can be used as a reference for scenarios such as e-commerce that need to process massive amounts of unstructured data. [6] [7]

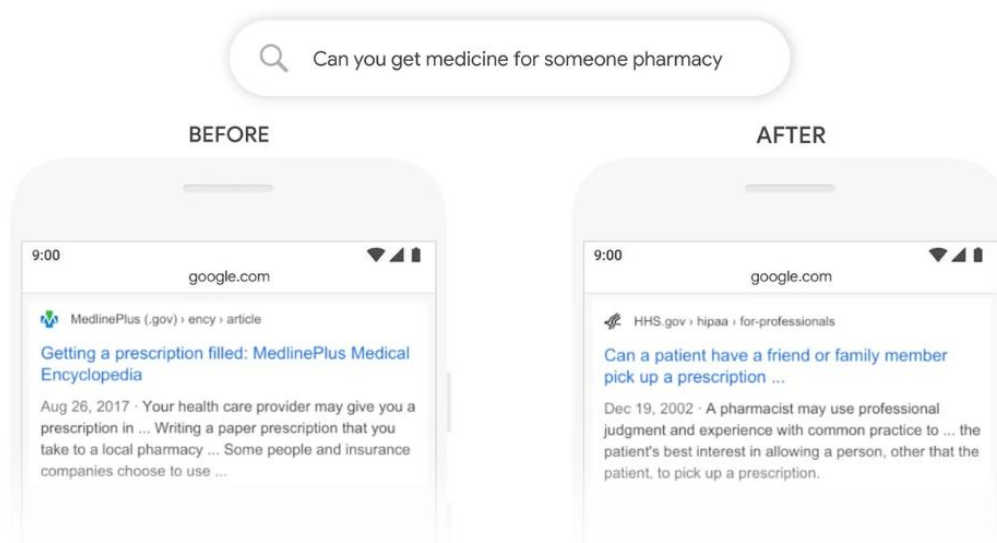


Figure 5 Understanding with BERT

2.2 Similarity Model

In e-commerce scenarios, the most direct role of similarity models is to "connect seemingly unrelated substitutes into a line": a sentence in the search box, a picture on the details page, and even a sentence in the comment area "This pair of shoes is a bit like AJ1 Retro OG" can all be mapped into vectors, and then the distance between vectors can be used to quickly find "likely the same style" and "similar style" substitutes. It not only drives the "Guess you like" on the homepage, but also supports linkage components such as "Look again and again" on the details page; during the promotion period, the platform will also use similarity models to de-duplicate inventory and spread hot products, push similar marketing resources to the most matching traffic pool, and shorten the cold start cycle of hot products. However, due to the current complexity of product information, relying solely on text or pictures to calculate similarity is often troubled by noise: gimmicks such as "new products" and "discounts" in the title or misleading descriptions of merchants will conceal the true style characteristics, and the lighting of product images, shooting angles, and differences in the patterns and styles of each product trademark will also greatly reduce the recognition effect of a single model. Cross-modal similarity compresses

"image-text" information into the same vector space, retaining language details while utilizing visual textures, and can more robustly capture implicit associations between "same type but different models" or "same function but different models". Currently commonly used mature solutions include VisualBERT, which combines regional features and word fragments into a single tower, an early alignment model based on BERT-ResNet fusion , UNITER (Chen et al., 2020), which performs joint mask learning on multiple corpora, and OpenCLIP , which achieves zero-shot retrieval through massive contrastive learning. Together, these methods have laid the foundation for the current mainstream practice of "cross-modal similarity + vector retrieval" in e-commerce recommendations.[8] [9] [10] [11]

CHAPTER 3. Project Execution Schedule

Throughout the semester, our project went through four major phases: data preprocessing, model training, model and strategy fine-tuning, and user interface development. Initially, we wanted to choose a project that could apply what we learned in class and try some widely used modern technologies. In the first few weeks, we exchanged ideas, explored various possibilities, and finally determined the direction, target results, and overall architecture of the project.

After confirming the feasibility, our attention turned to the data layer. Our product dataset (containing multiple CSV files, image URLs, and related details) was taken from an open source official website, but due to its rough content structure and extremely vague classification, its availability was better than nothing. We had to do a lot of processing: download all the images one by one, remove duplicate and abnormal data, and reorganize these resources into a consistent dataset. This stage also included reassigning or subdividing category labels and performing statistical tests on price and review distribution to ensure that the final dataset could handle the next complex tasks.

After about one and a half months of continuous data optimization, the dataset became relatively reliable. Subsequently, we began to train and fine-tune the model. The rapid development of the basic retrieval model provided preliminary search capabilities and laid the foundation for subsequent enhancements. However, the multimodal recommendation model proved to be more time-consuming and required a lot of computing resources. In the meantime, we tested and adjusted features and hyperparameters to optimize precision and recall. A week after the final exam, we built a model that performed relatively well.

Next, the focus shifted to the user interface. Within a week, we built a simple and easy-to-use layout. Within a month after the mid-term report, multimodal similarity recommendations and user-item collaborative recommendations were also added to the system as the manually constructed dataset took shape. But as the results were not satisfactory, we found some serious problems with the previous tasks. So we spent another month reworking, adjusting the dataset again, manually annotating and adjusting the reclassification strategy again and again, and retraining and fine-tuning all the subsequent models after verifying that they were reliable. Fortunately, we got correct and reliable results later. In the remaining time, we beautified the user interface, dealt with some hidden minor issues we found, and completed the entire task on schedule.

The bottom is our mid-term flow progression Gantt chart.

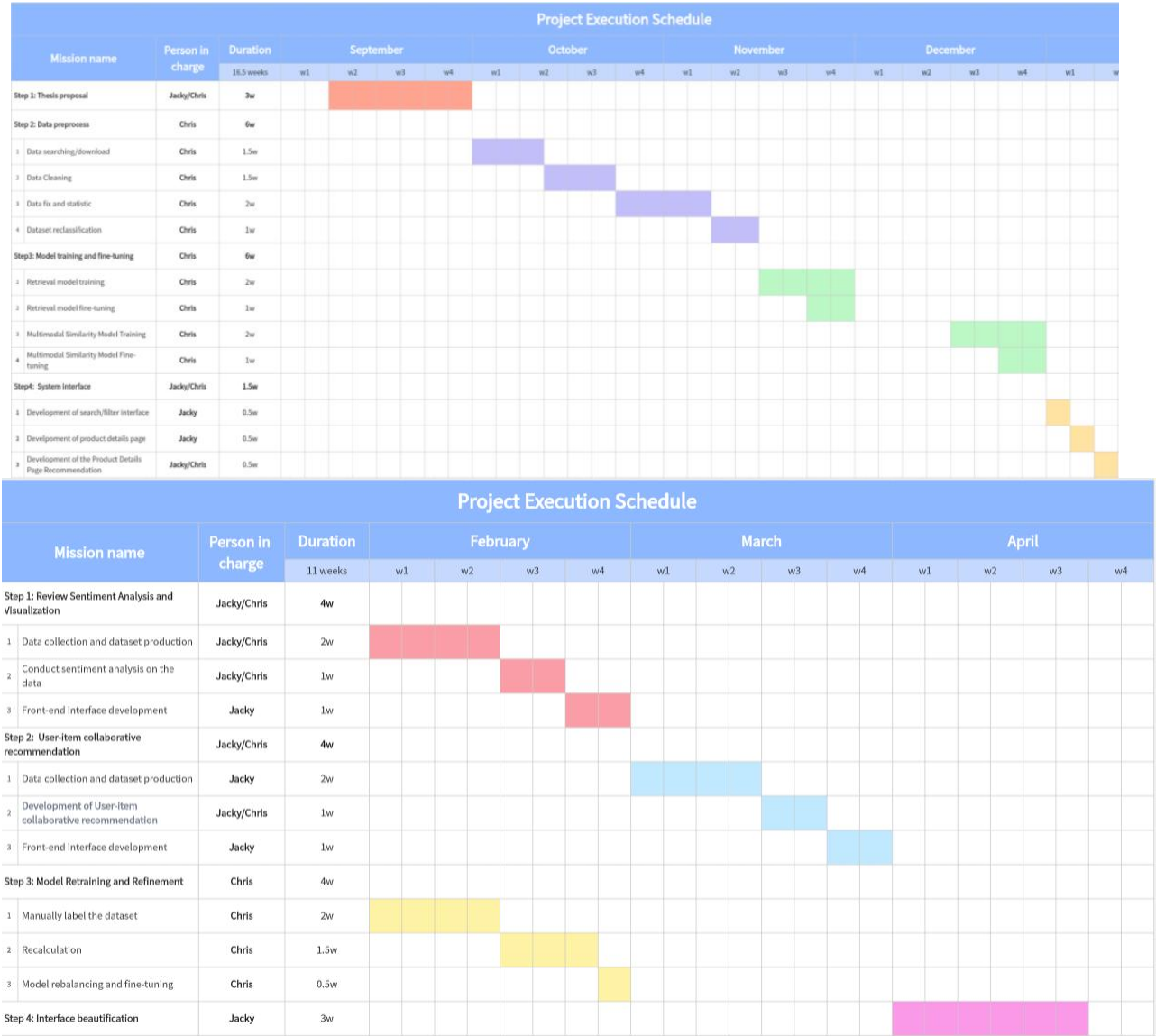


Figure 6. Gantt chart

CHAPTER 4. Method

In this chapter, I will introduce the methods for building product search and recommendation systems, following the main development stages of the project: data preprocessing, model training and fine-tuning, and the final user interface.

4.1 Data Preprocessing

For the dataset, we selected the Amazon UK dataset for 2023. This dataset covers data information of some products in that year, including price, title, product number, star rating, etc. [12][13]

4.1.1 Data Correction and Statistical Analysis

After completing the initial data download and cleaning, I conducted further statistical analysis and adjustments to ensure the dataset would be suitable for subsequent multimodal retrieval and classification tasks. In e-commerce scenarios, there is often a pronounced imbalance between “ best-selling categories ” and “ long-tail categories. ” Without addressing these discrepancies, the training process may inadvertently neglect certain categories or overfit to others. Therefore, as part of the statistical procedure, I counted and filtered items based on their categoryName. For any category whose item count was unusually large or small, I applied balanced sampling or down-sampling to maintain a more appropriate distribution across both the training and testing sets.

4.1.2 Dataset Reclassification

Beyond the issues of missing products or invalid image links, a major obstacle in the original dataset was the labeling of product categories.

Below is a sample excerpt from the initial dataset.

```
"B0CFVJKLQD","FGUUTYM Men's Tummy Control Underwear Breathable Traceless 3D Men's
Panties Pure Cotton Printed Boxer Shorts Sexy Boxer Shorts with Medium
Waist","https://m.media-
amazon.com/images/I/612cowNdwjL._AC_UL320_.jpg","https://www.amazon.co.uk/dp/B0CFVJKLQ
D","0.0","0","3.46","False","0","Sports & Outdoors"
```

```
"B0CCVJ6RW9","FGUUTYM Men's Fashion Loose Seams Fake Two Piece Hoodie Top Hoodie Boys
170","https://m.media-
amazon.com/images/I/5170bdIhXHL._AC_UL320_.jpg","https://www.amazon.co.uk/dp/B0CCVJ6RW
9","0.0","0","23.21","False","0","Sports & Outdoors"
```

```
"B0CCVFZSNK","CreoQIJI Hoodie Boys Autumn Winter Fashion Men Loose Pullover Hooded
Plus Size Sweatshirt Men Derbe","https://m.media-
amazon.com/images/I/612HteTrdQL._AC_UL320_.jpg","https://www.amazon.co.uk/dp/B0CCVFZSN
K","0.0","0","12.99","False","0","Sports & Outdoors"
```

The example shows that each record starts with product-related information (such as product code, title, star rating, and price) and ends with a category label. Unfortunately, these records are classified into a broad and imprecise category regardless of what the product actually is. This poses a serious challenge for supervised learning in the visual field, because each category no longer contains

unique features for the model to learn. Faced with tens of thousands of samples, manual reclassification is not feasible.

Therefore, I integrated the multimodal feature extraction process into my workflow. Specifically, I used Open AI's open source pre-trained CLIP model [14] to encode product titles (text encoding) and main product images (image encoding). The model labels and vectorizes each text title and extracts image features at the same time. In some cases, simple fusion or comparison is performed to capture other semantic clues.

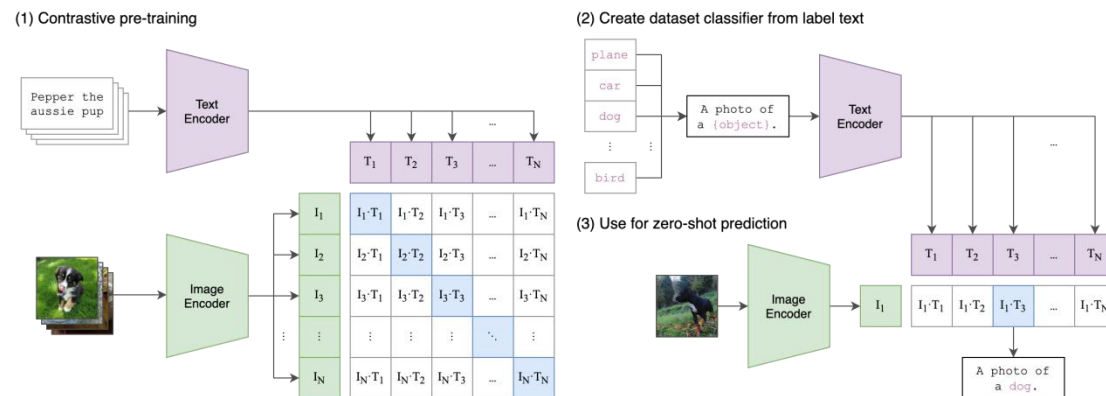


Figure 7 openCLIP architecture diagram

I first use CLIP to extract visual features from the product image and keep this step offline; the resulting caption is then concatenated with the product title and fed into Flan-T5 to produce finer-grained sub-category labels. [15]

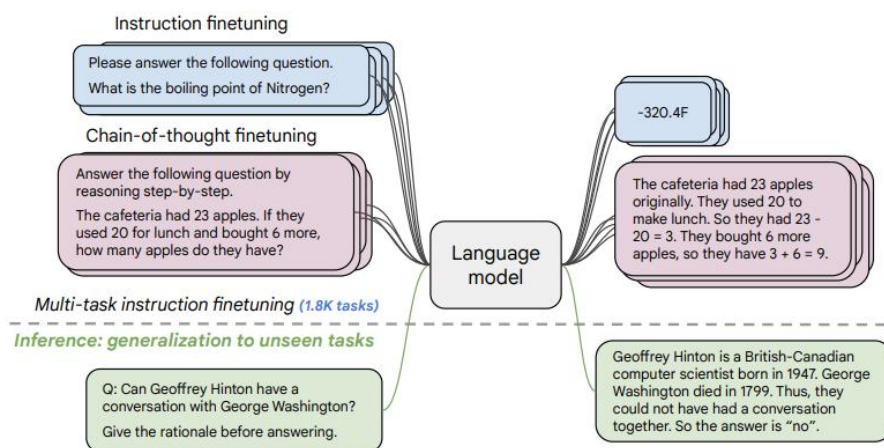


Figure 8 Flan-T5 architecture diagram

Because CLIP and Flan-T5 are applied sequentially with their own pre-trained weights, noise in images and ambiguities in titles can still introduce label drift, so we manually corrected a representative subset of classes. Therefore, I decided to extract some practical classes from the dataset and manually annotate them. Since the previously processed dataset is still huge and difficult to estimate as a whole, I have significantly reduced the available size of the dataset, including the number and categories. I chose the categories "Shirt", "Jacket", "Bike", "Pants", "Shorts", "Skirt", "Socks", "Boots", "Sneakers", "Slippers", and "Others" for implementation. It was observed that the graphic features of these categories are relatively comprehensive, and many categories are very similar in text descriptions and product image features. Therefore, if these easily confused categories can be correctly classified, then processing larger datasets will be a piece of cake. Sure enough, after the addition of the annotation set, the model classification accuracy was significantly improved, and it was basically reliable after manual verification.

4.2 Model Training and Fine-Tuning

Upon completion of data preparation, I focused on two core directions in model development: a retrieval model aimed at aligning text and images in a shared embedding space for efficient product searches, and a multimodal similarity model that further integrates textual and visual representations for more accurate classification and recommendation. The first model addresses the challenge of transforming text queries into comparable image vectors, while the second emphasizes deeper feature fusion to capture fine-grained product characteristics.

4.2.1 Retrieval Model

The primary objective of this retrieval model is to embed textual queries and product images into a shared vector space, where cosine similarity or inner-product distance gauges their semantic closeness. To accomplish this, I adopted a pretrained OpenCLIP network, originally trained via contrastive learning: one branch relies on a ViT to process images, and the other uses a Transformer to encode text inputs. During pretraining, matching image – text pairs are drawn closer in the latent space, while non-matching pairs are pushed apart. Such cross-modal alignment fits e-commerce searches particularly well, as users typically supply textual descriptions while the actual inventory is predominantly in image form. By leveraging an existing OpenCLIP model, the system inherits broad generalization across diverse product visuals and descriptions, thus reducing the likelihood that users will overlook items matching their intentions.

Before training or inference, images undergo basic transformations — resizing and normalization—alongside random flips or crops to counter overfitting to any specific viewpoint. Text data receive a similar treatment through the built-in OpenCLIP tokenizer. These moderate augmentations help the model robustly handle real-world variations in lighting and camera angles found in product images. Since the training set includes nearly 160,000 samples, it is not feasible to assign multiple images per product; this augmentation strategy is thus partly a compromise that expands data diversity without exponential overhead.

When minor domain shifts appear—such as unique phrasing in product titles—the model can be fine-tuned for several epochs under a project-specific contrastive objective. By maximizing similarity for true image – text pairs while minimizing it for mismatched ones, the system adapts more closely to the dataset’s real-world distribution, narrowing any discrepancy introduced by pretraining on large, generic corpora.

Once embeddings are obtained from the retrieval model, Faiss is used for vector-based search and indexing. Faiss supports high-throughput nearest-neighbor queries on GPUs, a practical option for mid- to large-scale e-commerce platforms. Searching by text simply becomes a matter of encoding the user’s query and retrieving the most relevant image vectors from the Faiss index. Although some redundancies and algorithmic simplifications still make response times perceptible, overall latency remains within an acceptable range, providing a responsive and scalable retrieval pipeline.

4.2.2 Multimodal Similarity Model

Although contrastive alignment of embeddings is already sufficient for cross-modal product searches, I also employ a commonly used multimodal similarity model to recommend similar items. By unifying image and text representations at a deeper level, this approach captures subtle features—such as material, style, or fine-grained design elements—that may not be fully distinguishable through CLIP alone. These attributes play a pivotal role in product classification and fine-grained recommendations.

From an architectural standpoint, the model adopts a dual-stream layout: one pathway processes images via ResNet50 [16], the other pathway encodes text with BERT [17].

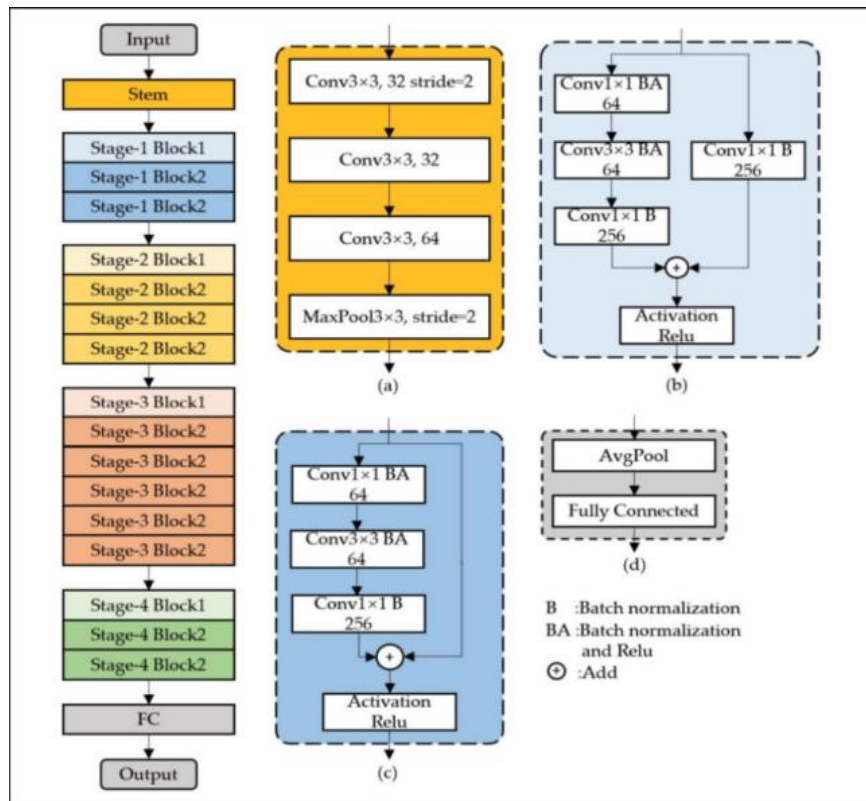


Figure 9. Resnet50 architecture diagram

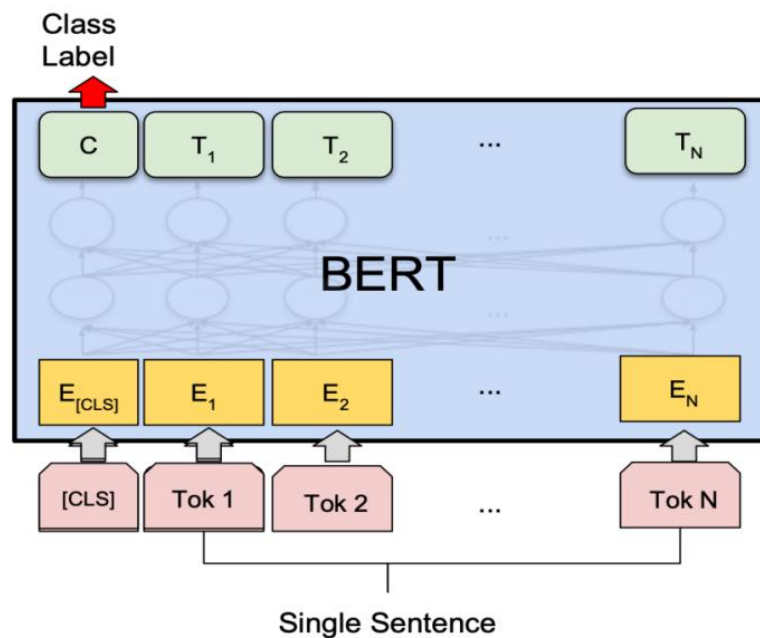


Figure 10. BERT architecture diagram

ResNet50' s skip connections mitigate gradient vanishing in deeper layers, allowing it to effectively extract visual features, whereas BERT' s strong text embeddings and

robust generalization capacity make it a suitable option for handling product descriptions. Although I chose not to unfreeze additional layers in pursuit of higher performance, the current configuration adequately meets the project's demands. A fusion layer then combines these two representations using multi-head attention or a more straightforward weighted averaging scheme, thereby surfacing cross-modal interactions. This step is particularly helpful in differentiating nearly identical products that vary only in subtle details, leading to improved classification accuracy or item-to-item similarity.

Data augmentation remains crucial to the pipeline. Random cropping or color jitter are applied to images, and occasional spelling irregularities or special characters in text can be mitigated by moderate preprocessing or noise injection. The choice of training objectives depends on system goals: for classification tasks, I rely on cross-entropy or focal loss — especially beneficial in the face of highly skewed class distributions—whereas for similarity-based tasks, contrastive or multi-instance losses facilitate clustering embeddings of semantically related items while separating those that are dissimilar.

$$\text{Cross-entropy loss : } L_{CE} = - \sum (y_i * \log(p_i))$$

$$\text{Focal loss: } L_{FL} = - \alpha_i * (1 - p_i)^\gamma * \log(p_i)$$

To further enhance performance, I employ AdamW as an optimizer along with a learning rate scheduler like ReduceLROnPlateau, which automatically lowers the learning rate once validation metrics stabilize.

AdamW sample :

Phase 1: Update of first-order and second-order momentum

$$m_t = \beta_1 * m_{(t-1)} + (1 - \beta_1) * g_t$$

$$v_t = \beta_2 * v_{(t-1)} + (1 - \beta_2) * (g_t)^2$$

Bias-Correction

$$m_{\hat{}} = m_t / (1 - \beta_1^t)$$

$$v_{\hat{}} = v_t / (1 - \beta_2^t)$$

Phase 2: Parameter update (including weight decay)

$$w_{(t+1)} = w_t - lr * (m_{\hat{}} / (\sqrt{v_{\hat{}}} + \epsilon) + \lambda * w_t)$$

After continuous optimization, this multimodal similarity model has become even smarter. It can not only more accurately extract the semantics behind each product,

making the classification more precise, but more importantly, the recommendations given are more in line with what users want. This deeply integrated embedding technology makes the shopping experience particularly natural. The system no longer lists products mechanically as before. In actual use, it can effectively avoid various misjudgment problems caused by simple statistics or keyword matching.

CHAPTER 5. Implementation (Individual)

In the previous chapter, I have shown the implementation methods of several functional modules. Now, I will further show the implementation of these methods, that is, the specific script structure and code snippets, and explain how these codes implement the functions mentioned above.

5.1 Code Structure Overview

Data preprocessing is managed by multiple Python scripts, each addressing a different stage:

downloadimages

Reads product information and image URLs from the original CSV file (*amz_uk_processed_data.csv*).

Sample

```
"B0BL2G73K3","Shoulder bag for Men Crossbody Small Shoulder Bag Portable mobile phone
coin bag/Travel Utility Work Practical Handy Mens","https://m.media-
amazon.com/images/I/61NhlwtqFDL._AC_UL320_.jpg","https://www.amazon.co.uk/dp/B0BL2G73K
3","4.4","299","9.98","False","0","Men"
```

It then uses HTTP requests to download the images locally and updates the CSV so that each product record references its newly saved image filename.

Sample

```
B0BL2G73K3,Shoulder bag for Men Crossbody Small Shoulder Bag Portable mobile
phone coin bag/Travel Utility Work Practical Handy
Mens,images/val/val_1540665.jpg,https://www.amazon.co.uk/dp/B0BL2G73K3,4.4,299,
9.98,False,0, Men
```

clean / num

Verifies that each downloaded image file indeed corresponds to a valid record in the CSV, filtering out any invalid or non-existent entries to maintain data consistency. After these preliminary steps, I output the category distribution, providing an overview for subsequent decisions.

Sample (categoryName)

Sports & Outdoors	660860
Skin Care	15004
.....	
Heating, Cooling & Air Quality	3575
Professional Education Supplies	3572

```
return pd.DataFrame(processed_rows)
```

reclassification

Leverages models such as CLIP to perform reclassification on product images and text descriptions, providing more detailed or appropriately labeled categories compared to the original broad ones.

```
# path
train_file = r"C:\Users\User\Desktop\FYP\dataset\train_labeled.csv"
val_file = r"C:\Users\User\Desktop\FYP\dataset\val.csv"
base_dir = r"C:\Users\User\Desktop\FYP\dataset"
output_file = r"C:\Users\User\Desktop\FYP\dataset\val_updated_final.csv"
category_counts_output = r"C:\Users\User\Desktop\FYP\dataset\val_category_counts.csv"

# set category
candidate_categories = [
    "Shirt", "Jacket", "Bike", "Pants", "Shorts", "Skirt", "Socks", "Boots",
    "Sneakers", "Slippers", "Others"
]

# load CLIP
clip_device = "cuda" if torch.cuda.is_available() else "cpu"
model_name = "openai/clip-vit-base-patch32"
clip_model = CLIPModel.from_pretrained(model_name).to(clip_device)
clip_processor = CLIPProcessor.from_pretrained(model_name)

# Extracting image features
def extract_image_features(image_paths):
    images = []
    for path in image_paths:
        full_path = os.path.join(base_dir, path)
        try:
            img = Image.open(full_path).convert("RGB")
        except:
            img = Image.new("RGB", (224, 224))
        images.append(img)
    inputs = clip_processor(images=images, return_tensors="pt",
padding=True).to(clip_device)
    with torch.no_grad():
        features = clip_model.get_image_features(**inputs)
    return features.cpu().numpy()

# Obtain multimodal features
def get_multimodal_features(df, img_weight=0.4, text_weight=0.6):
    image_features = extract_image_features(df["imgUrl"].tolist())

    text_inputs = clip_processor(
        text=df["title"].fillna("").tolist(),
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=77
    ).to(clip_device)

    with torch.no_grad():
        text_features = clip_model.get_text_features(**text_inputs).cpu().numpy()

    multimodal_features = (image_features * 0.4) + (text_features * 0.6)
    return multimodal_features

# Train classifier (fine-tune)
def train_classifier(train_df):
    train_df["categoryName"] = train_df["categoryName"].str.strip().str.title()
    train_df.loc[~train_df["categoryName"].isin(candidate_categories),
"categoryName"] = "Others"
```

```

multimodal_features = get_multimodal_features(train_df)
labels = train_df["categoryName"].apply(lambda x:
candidate_categories.index(x)).values
clf = LogisticRegression(max_iter=1000)
clf.fit(multimodal_features, labels)
return clf

# Classification Validation Data
def classify_val_data(val_df, clf, threshold=0.5):
    predicted_categories = []

    for _, row in tqdm(val_df.iterrows(), total=len(val_df), desc="Classifying
Validation Data"):
        single_row_df = pd.DataFrame([row])
        features = get_multimodal_features(single_row_df)
        probabilities = clf.predict_proba(features)
        pred_idx = probabilities.argmax(axis=1)[0]
        max_prob = probabilities.max(axis=1)[0]

        category = candidate_categories[pred_idx] if max_prob >= threshold else
"Others"
        predicted_categories.append(category)

    val_df["predicted_category"] = predicted_categories
    return val_df

```

At this point, most samples are ready for subsequent tasks.

5.2 Model Training and Finetuning

After acquiring and organizing the data, my main task is to call, train and fine-tune two models: the retrieval model (represented by OpenCLIP + Faiss) and the multimodal similarity model (ResNet50 + BERT). The following will show the main scripts and implementation methods of each.

5.2.1 Retrieval Model (Retrieval Model: OpenCLIP + Faiss)

In short, the core idea of this model is to map product images and text descriptions to the same vector space (through OpenCLIP comparative learning), and then use Faiss to establish vector indexes for fast search.

```

def build_faiss_index(model, preprocess, tokenizer, csv_files, index_path):
    frames = [pd.read_csv(f) for f in csv_files]
    combined_df = pd.concat(frames, ignore_index=True)

    items_meta = []
    embeddings = []

    for idx, row in combined_df.iterrows():
        img_path = os.path.join("images", row["imgUrl"])
        if not os.path.exists(img_path):
            continue # skip invalid images
        try:
            image = Image.open(img_path).convert("RGB")
            img_tensor = preprocess(image).unsqueeze(0).to("cuda")

            with torch.no_grad():
                img_features = model.encode_image(img_tensor)
                img_features /= img_features.norm(dim=-1, keepdim=True)

            meta = {
                "title": row["title"],
                "price": row["price"],
                "stars": row["stars"],
                "imgUrl": row["imgUrl"],
            }

```

```

        items_meta.append(meta)
        embeddings.append(img_features.cpu().numpy()[0])
    except:
        continue

embeddings = np.array(embeddings, dtype=np.float32)
d = embeddings.shape[1]
index = faiss.IndexFlatIP(d)
index.add(embeddings)

faiss.write_index(index, index_path)
with open("items_meta.json", "w") as f:
    json.dump(items_meta, f)

print(f"Index built with {len(embeddings)} items.")

```

After building the index, the query function only needs to encode the text into a vector and then call the nearest neighbor search in the index. This greatly improves the efficiency of the system.

```

def search_products(query, model, tokenizer, index, items_meta, top_k=5):
    model.eval()
    text_tokens = tokenizer([query]).to("cuda")
    with torch.no_grad():
        text_features = model.encode_text(text_tokens)
        text_features /= text_features.norm(dim=-1, keepdim=True)

    text_numpy = text_features.cpu().numpy().astype(np.float32)
    distances, indices = index.search(text_numpy, top_k)

    results = [(items_meta[i], distances[0][k]) for k, i in
               enumerate(indices[0])]
    return results

```

User input, such as "red shoes", is encoded into a text vector by CLIP, and `index.search()` returns the image products with the highest similarity, thus realizing "text → text/image" cross-modal retrieval.

5.2.2 Multimodal Similarity Model (ResNet + BERT)

In terms of recommendation function, I implemented a recommendation model based on image-text similarity. In this part, I adopted a dual-stream architecture: ResNet50 extracts image features, BERT extracts text features, and then does some slightly crude fusion at the high level.

1. Extract image features

First, use ResNet-50 to remove the last fully connected layer to obtain a 2048-dimensional global vector, then map it to 512 dimensions and add -Dropout.

```

self.resnet = nn.Sequential(
    *list(resnet50(weights=ResNet50_Weights.DEFAULT).children())[:-1]
)
self.fc1 = nn.Linear(2048, 512)
self.dropout = nn.Dropout(0.5)

img_features = self.resnet(image).view(image.size(0), -1) # (B, 2048)
img_features = torch.relu(self.fc1(img_features)) # (B, 512)
img_features = self.dropout(img_features)

```

2. Extract text features

Use BERT to obtain the [CLS] vector (768 dimensions), and then map it to 512 dimensions.

```
self.bert = BertModel.from_pretrained('bert-base-uncased')
self.fc2 = nn.Linear(768, 512)

text_output = self.bert(input_ids, attention_mask=attention_mask)
text_features = torch.relu(self.fc2(text_output.last_hidden_state[:, 0, :]))
# (B, 512)
```

3. Splice into 2-token sequence

The image and text are each considered as a token, which are concatenated along the sequence dimension and sent to the attention layer.

```
img_features = img_features.unsqueeze(1) # (B, 1, 512)
text_features = text_features.unsqueeze(1) # (B, 1, 512)
combined = torch.cat((img_features, text_features), dim=1) # (B, 2, 512)
```

4. Cross-modal multi-head attention fusion

Use nn.MultiheadAttention to make two tokens pay attention to each other, and then average the two tokens to get the fusion vector.

```
self.attention = nn.MultiheadAttention(embed_dim=512, num_heads=8,
batch_first=True)

attn_output, _ = self.attention(combined, combined, combined) # (B, 2, 512)
fused_features = attn_output.mean(dim=1) # (B, 512)
```

5. Classification output

The final 512-dimensional fusion vector is sent to the fully connected layer to output the logits of each category.

```
self.fc3 = nn.Linear(512, num_classes)

output = self.fc3(fused_features) # (B, num_classes)
return output
```

5.3 Recommendation

5.3.1 Similarity-Based Rec

See 5.2.2 for models and methods

5.3.2 User-Item Interaction Rec

In the review analysis module, I used spaCy and VADER to analyze user reviews and convert sentiment into intuitive scores. Interestingly, we found that users who bought

the same product often had similar tastes, which is why the "recommendations for like-minded people" feature came about - it not only recommends products in the same category, but also recommends other popular products across categories. The word-of-mouth analysis feature can automatically extract the most frequently mentioned advantages and disadvantages from a large number of reviews, allowing users to get the main selling points and potential problems of the product at a glance.

```
# Load NLP models
nlp = spacy.load("en_core_web_sm")
analyzer = SentimentIntensityAnalyzer()

def sentiment_analysis(review):
    sentiment_score = analyzer.polarity_scores(review)['compound']
    if sentiment_score < 0:
        return -1 # Negative
    elif sentiment_score < 0.5:
        return 0 # Neutral
    else:
        return 1 # Positive

data['Rating'] = data['Review'].apply(sentiment_analysis)

def recommend_based_on_category(item_id, top_n=3):
    users_who_bought_item = data[data['ASIN'] == item_id]['user'].tolist()

    other_items_bought_by_users = data[data['user'].isin(users_who_bought_item)]

    other_items_bought_by_users_with_category =
other_items_bought_by_users[['ASIN', 'Production']]

    target_item_category = data[data['ASIN'] == item_id]['Production'].iloc[0]

    other_category_items = other_items_bought_by_users_with_category[
        other_items_bought_by_users_with_category['Production'] !=
target_item_category
    ]

    item_counts = Counter(other_category_items['ASIN'])

    recommended_items = [item for item, count in item_counts.most_common() if
item != item_id]

    return recommended_items[:top_n], target_item_category

def recommend_random_items_from_category(category, top_n=3):
    same_category_items = category_data[category_data['categoryName'] ==
category]

    if len(same_category_items) < top_n:
        recommended_items = same_category_items['asin'].tolist()
    else:
        recommended_items = random.sample(same_category_items['asin'].tolist(),
top_n)

    return recommended_items

def get_item_composite_score(item_id):
    return data[data['ASIN'] == item_id]['Rating'].mean()

def get_positive_negative_adjectives(item_id):
    adjective_summary = get_item_review_summary(item_id)
    positive_adjectives = []
    negative_adjectives = []

    for adjective, freq in adjective_summary.items():
        sentiment_score = analyzer.polarity_scores(adjective)['compound']
        if sentiment_score > 0:
            positive_adjectives.append((adjective, freq))
        elif sentiment_score < 0:
            negative_adjectives.append((adjective, freq))
```

```

        positive_adjectives = sorted(positive_adjectives, key=lambda x: x[1],
reverse=True)[:3]
        negative_adjectives = sorted(negative_adjectives, key=lambda x: x[1],
reverse=True)[:3]

        return positive_adjectives, negative_adjectives

def get_item_review_summary(item_id):
    item_reviews = data[data['ASIN'] == item_id]
    adjectives = []
    for review in tqdm(item_reviews['Review'], desc=f"Processing reviews for
{item_id}"):
        adjectives.extend(extract_adjectives(review))
    return Counter(adjectives)

def extract_adjectives(review):
    """Extract adjectives from text using spaCy"""
    doc = nlp(review.lower())
    return [token.text for token in doc if token.pos_ == 'ADJ']

```

5.4 UI and User Interaction

After the data and model are ready, an intuitive interface is needed for users. To this end, we use CustomTkinter to implement the desktop graphical interface, integrating functions such as search bar, filter control, result list, product details and recommendation area. This Tkinter-based extension library provides more theme support (such as dark/light mode), more customizable rounded corner controls, and improved scrollable containers, etc., which is closer to modern GUI in terms of visual and usability.

```

class ProductSearchSystem:
    def __init__(self):
        self.SEARCH_DIR = os.environ.get('PRODUCT_SEARCH_DIR',
r'D:\FYP\FYP\Search')
        self.INDEX_PATH = os.path.join(self.SEARCH_DIR, "faiss_index.index")
        self.META_JSON = os.path.join(self.SEARCH_DIR, "items_meta.json")
        self.EMB_PATH = os.path.join(self.SEARCH_DIR, "embeddings_sim.npy")

        self.MODEL_NAME = "ViT-L-14"
        self.PRETRAINED = "openai"
        self.DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
        print(f"Using device: {self.DEVICE}")
        self.TOP_K = 15

        self.model = None
        self.preprocess = None
        self.tokenizer = None
        self.index = None
        self.items_meta = []
        self.embeddings = None
        self.load_success = False

        try:
            self.init_model()
            self.load_data()
            self.load_success = True
            # Create ASIN to index mapping for faster lookups
            self.asin_to_metadata_map = {item.get('asin') or item.get('ASIN'):
item
                                for item in self.items_meta
                                if item.get('asin') or
item.get('ASIN')}
            print(f"Created ASIN to metadata map with
{len(self.asin_to_metadata_map)} entries.")

```

```

except Exception as e:
    print("="*30)
    print("!!! FAILED TO INITIALIZE ProductSearchSystem !!!")
    print(f"Error: {e}")
    print("Please check model availability, file paths, and
dependencies.")
    print("Search functionality will be disabled.")
    print("="*30)
    traceback.print_exc()

def init_model(self):
    print(f"Initializing CLIP model: {self.MODEL_NAME}
({self.PRETRAINED})...")
    self.model, _, self.preprocess = open_clip.create_model_and_transforms(
        self.MODEL_NAME,
        pretrained=self.PRETRAINED,
        device=self.DEVICE
    )
    self.tokenizer = open_clip.get_tokenizer(self.MODEL_NAME)
    self.model.eval()
    print("Successfully initialized model and tokenizer.")

def load_data(self):
    print("Loading search index, metadata, and embeddings...")
    if not os.path.exists(self.INDEX_PATH):
        raise FileNotFoundError(f"Faiss index not found:
{self.INDEX_PATH}")
    self.index = faiss.read_index(self.INDEX_PATH)
    print(f"Loaded Faiss index with {self.index.ntotal} vectors.")

    if not os.path.exists(self.META_JSON):
        raise FileNotFoundError(f"Metadata JSON not found:
{self.META_JSON}")
    with open(self.META_JSON, 'r', encoding='utf-8') as f:
        self.items_meta = json.load(f)
    print(f"Loaded metadata for {len(self.items_meta)} items.")

    if not os.path.exists(self.EMB_PATH):
        raise FileNotFoundError(f"Embeddings file not found:
{self.EMB_PATH}")
    self.embeddings = np.load(self.EMB_PATH).astype('float32')
    print(f"Loaded embeddings array with shape: {self.embeddings.shape}")

    if self.index.ntotal != len(self.items_meta) or self.index.ntotal !=
self.embeddings.shape[0]:
        print("Warning: Mismatch between index size, metadata count, and
embeddings count!")
        print(f"Index: {self.index.ntotal}, Meta: {len(self.items_meta)},
Embeddings: {self.embeddings.shape[0]}")
        min_count = min(self.index.ntotal, len(self.items_meta),
self.embeddings.shape[0])
        print(f"Will proceed using the minimum count: {min_count}")
        print("Successfully loaded search data.")

def search(self, query):
    if not self.load_success:
        print("Search system failed to load. Search unavailable.")
        return []
    try:
        with torch.no_grad():
            text = self.tokenizer([query]).to(self.DEVICE)
            text_features = self.model.encode_text(text)
            text_features = text_features / text_features.norm(dim=-1,
keepdim=True)
            text_features = text_features.cpu().numpy().astype('float32')
            distances, indices = self.index.search(text_features, self.TOP_K)
            return [(self.items_meta[idx], float(dist)) for dist, idx in
zip(distances[0], indices[0]) if 0 <= idx < len(self.items_meta)]
    except Exception as e:
        print(f"Search error: {e}")
        traceback.print_exc()
        return []

def find_similar_items(self, item_idx: int, top_n=3):
    if not self.load_success:
        print("Search system failed to load. Similarity search
unavailable.")

```

```

        return []
    if item_idx < 0 or item_idx >= len(self.embeddings):
        print(f"Error: Invalid item index {item_idx}")
        return []
    if self.index is None or self.embeddings is None:
        print("Error: Index or embeddings not loaded.")
        return []

    query_emb = self.embeddings[item_idx : item_idx+1].copy()
    faiss.normalize_L2(query_emb)
    try:
        distances, indices = self.index.search(query_emb, top_n + 1)
        results = []
        max_valid_index = len(self.items_meta)
        for dist, idx in zip(distances[0], indices[0]):
            if idx == item_idx: continue
            if not (0 <= idx < max_valid_index): continue
            item = self.items_meta[idx]
            results.append((item, dist))
            if len(results) >= top_n: break
        return results
    except Exception as e:
        print(f"Error finding similar items: {e}")
        traceback.print_exc()
        return []

def get_metadata_by_asin(self, asin: str):
    if not self.load_success:
        return None
    return self.asin_to_metadata_map.get(asin)

```

At this point, the search model and recommendation model are formally integrated into the system interface and begin to play their roles.

CHAPTER 6. Result (testing)

6.1 Terminal Output:

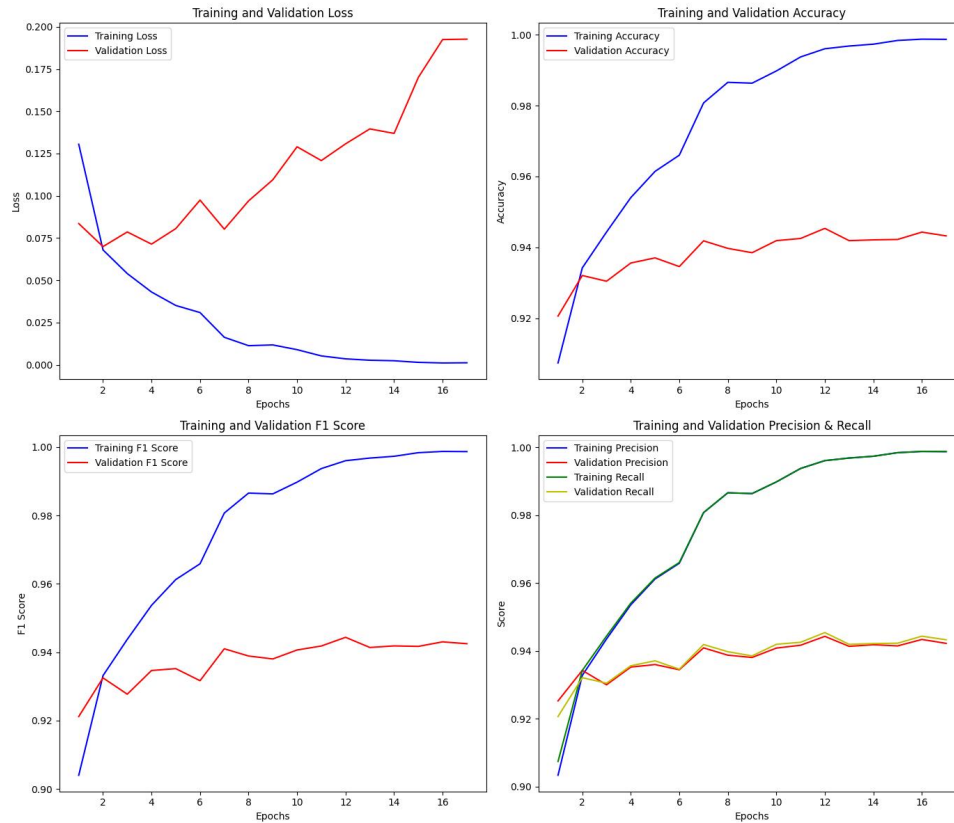


Figure 11 Model training fitting curve

```

2025-03-11 01:12:28,146 INFO:Number of classes: 11
2025-03-11 01:12:28,149 INFO:Singleton classes (only 1 sample): []
2025-03-11 01:12:41,901 INFO:All image files in Training Set exist.
2025-03-11 01:12:43,458 INFO:All image files in Validation Set exist.
2025-03-11 01:12:43,774 INFO:Tokenizer initialized.
2025-03-11 01:12:43,816 INFO:Datasets created.
2025-03-11 01:12:43,817 INFO>Data loaders created.
2025-03-11 01:12:45,073 INFO:Model initialized. Using device: cuda
2025-03-11 01:38:12,322 INFO:Epoch 1/30 Summary:
2025-03-11 01:38:12,322 INFO:Train Loss: 0.1305 | Accuracy: 0.9074 | Precision:
0.9033 | Recall: 0.9074 | F1 Score: 0.9040
2025-03-11 01:38:12,323 INFO:Val Loss: 0.0836 | Accuracy: 0.9206 | Precision:
0.9252 | Recall: 0.9206 | F1 Score: 0.9212

2025-03-11 01:38:12,836 INFO:--> Saved Best Model (Epoch 1)

2025-03-11 02:03:46,324 INFO:Epoch 2/30 Summary:
2025-03-11 02:03:46,325 INFO:Train Loss: 0.0680 | Accuracy: 0.9342 | Precision:
0.9327 | Recall: 0.9342 | F1 Score: 0.9332
2025-03-11 02:03:46,325 INFO:Val Loss: 0.0699 | Accuracy: 0.9321 | Precision:
0.9342 | Recall: 0.9321 | F1 Score: 0.9325

2025-03-11 08:00:52,995 INFO:No improvement in F1 Score for 4 epoch(s).
2025-03-11 08:26:24,997 INFO:Epoch 17/30 Summary:
2025-03-11 08:26:24,997 INFO:Train Loss: 0.0012 | Accuracy: 0.9987 | Precision:
0.9987 | Recall: 0.9987 | F1 Score: 0.9987

```

```
2025-03-11 08:26:24,997 INFO:Val Loss: 0.1927 | Accuracy: 0.9432 | Precision:
0.9422 | Recall: 0.9432 | F1 Score: 0.9425
```

```
2025-03-11 08:26:24,997 INFO:No improvement in F1 Score for 5 epoch(s).
2025-03-11 08:26:24,998 INFO:Early stopping triggered.
2025-03-11 13:15:55,671 INFO:Loaded the best model.
2025-03-11 13:18:11,906 INFO:Test Summary:
2025-03-11 13:18:11,906 INFO:Test Loss: 0.1301 | Accuracy: 0.9460 | Precision:
0.9448 | Recall: 0.9460 | F1 Score: 0.9449
```

6.2 Samples

Category Name		Accuracy	Train Samples
Test Samples	Score		

Others		0.9971	126768
15895	9.6458		
Shirt		0.9813	9501
1228	6.9807		
Skirt		0.9745	4400
549	6.1490		
Socks		0.9877	3292
407	5.9374		
Jacket		0.9846	2856
325	5.6979		
Sneakers		0.9872	2702
312	5.6725		
Slippers		0.9796	2316
294	5.5709		
Pants		0.9809	2299
262	5.4658		
Shorts		0.9764	1835
254	5.4104		
Boots		0.9892	1631
186	5.1749		
Bike		0.9943	1524
174	5.1351		

6.3 System :

As shown in the figure below, after entering the system, the main interface will be displayed to the user. The main part is some popular products with high sales and high ratings in the data set. For those shopping enthusiasts who "just browse casually", this is undoubtedly an eye-opener. But for users with clear categories, these displays may not be very helpful. Therefore, we set up a filter bar where users can select specific categories. If there is no favorite product on the current page, perhaps the "load more" button at the bottom can bring users more diverse and rich choices. Above the popular product recommendation area is a search bar for users to search for their target products. Below the search bar is an adjustable price range bar. By dragging the left and right orange endpoints, users can filter the product prices in the recommendation area below and the search list.

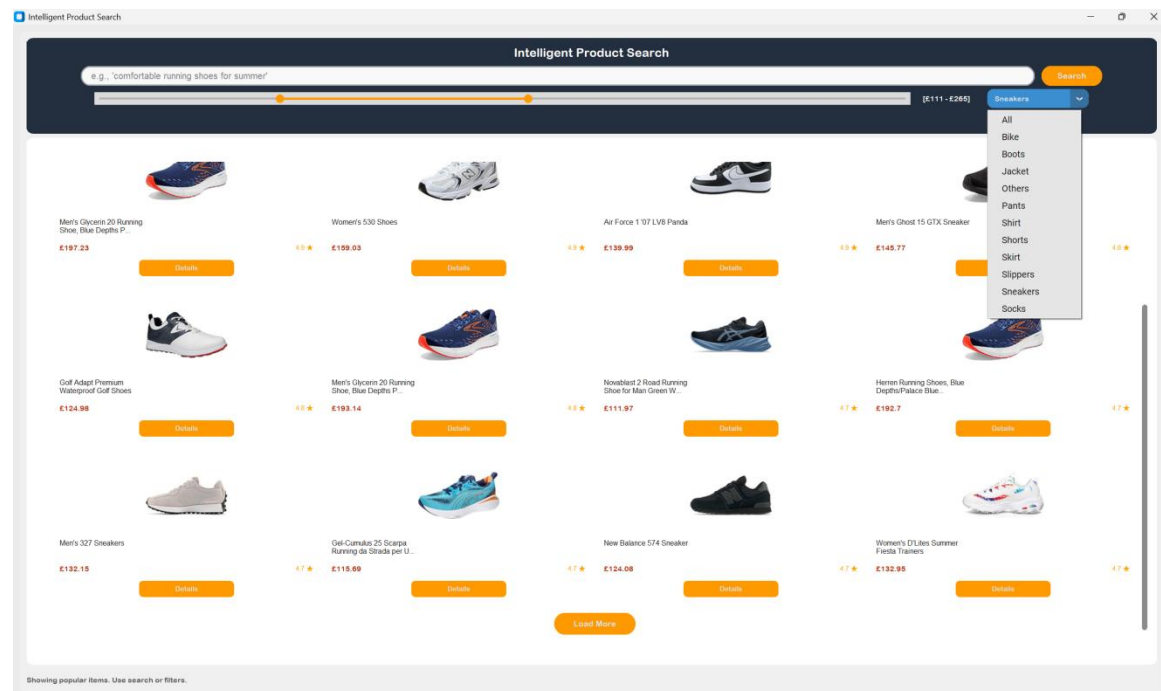


Figure 12 UI(General Browsing)

As shown below, the user describes his target product in the search bar. The user can enter in a relatively casual form. Every time the user presses the space bar, the target product list below will be updated until all qualifiers are entered, or before the input is completed, it is suddenly found that the desired product has been found (this is of course what we expect to see). The newly added filter button on the left side of the search interface can help users sort the list, including ascending and descending price, descending match, and descending score. The match is flattened from the vector similarity into a percentage, giving users a clearer and more direct reference. The "product link" button on the right side of the product card is used to jump directly to the corresponding product image on the UK Amazon official website (to avoid some possible disputes). The "View Detail" button will take the user to the next section.

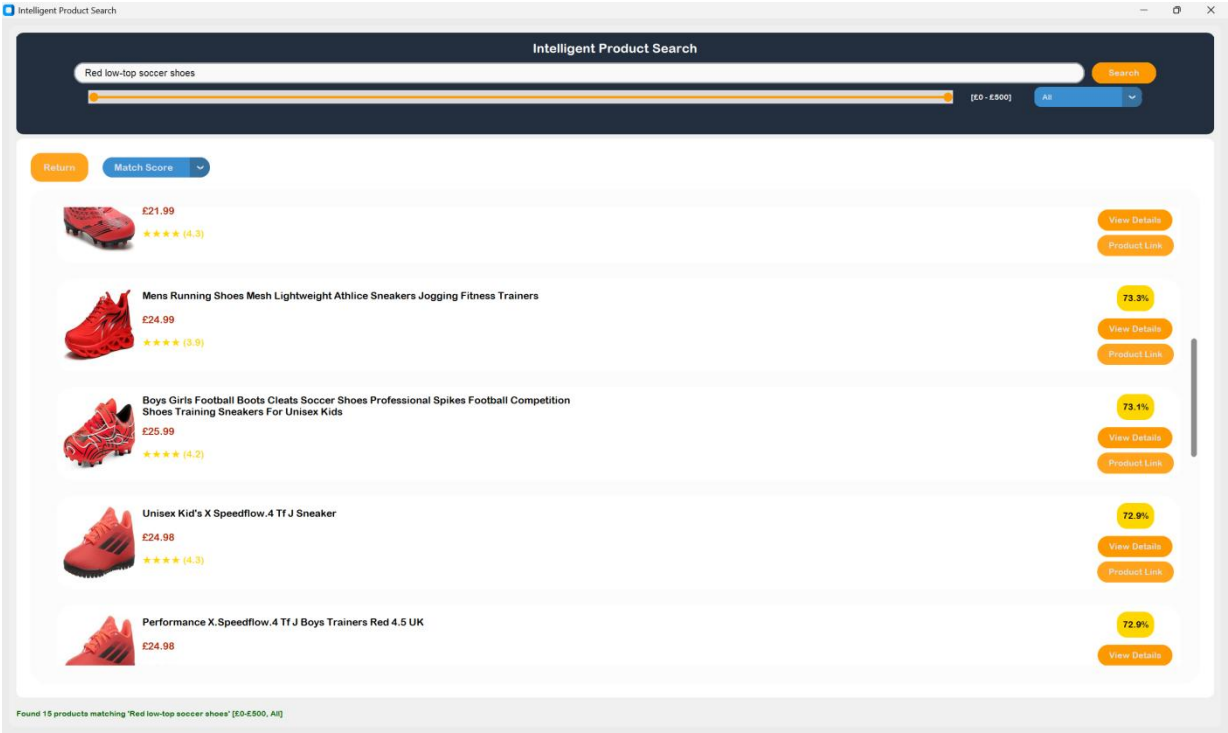


Figure 13 UI(Search)

Entering the product details page, users can intuitively browse the product pictures, text descriptions, prices, and categories. The comments of users who have already purchased the product are processed by the spaCy English model and the VADER (Valence Aware Dictionary and sEntiment Reasoner) sentiment tool, and a few positive and negative adjectives will be left, which will be displayed in the word cloud radar in the form of frequency. (During our design stage, most online platforms had not yet launched this function until deepseek became popular.) This will save viewers a lot of time reading comments. And it can also effectively filter out those water army who only give five stars but refuse to comment even a letter.

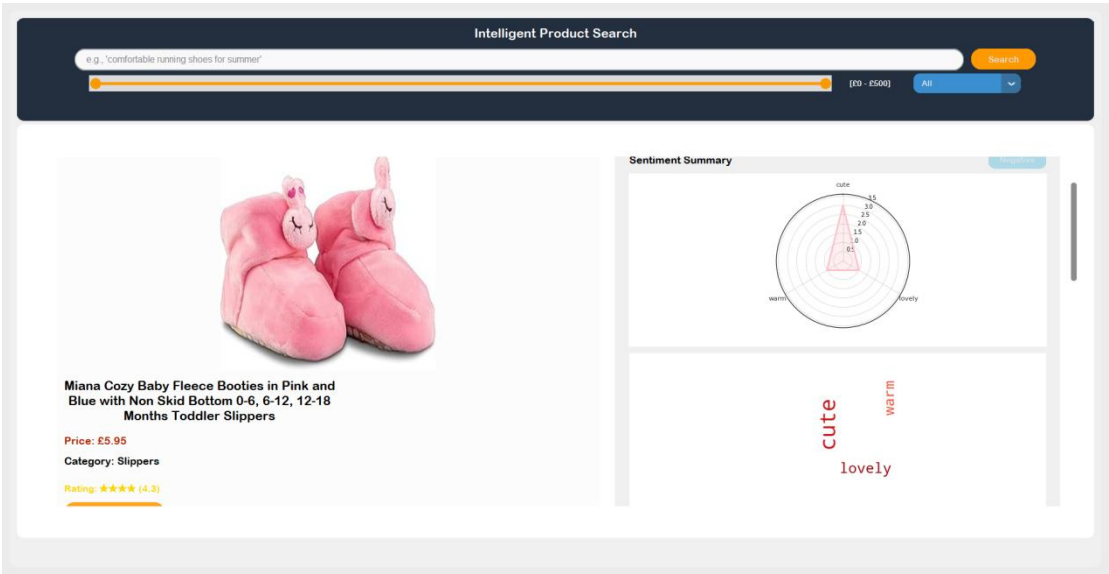


Figure 14 UI (product details)

With the organic integration of image and text similarity, the three most similar products are displayed to the user. Cross-modal similarity effectively breaks through the limitations of design factors such as brand and pattern while ensuring that the core product features remain unchanged, giving users more opportunities to make choices (perhaps users will find that the recommended products are almost identical but have very different prices due to brand differences, or perhaps users can find more favorite styles and colors by constantly browsing recommendations, etc.)

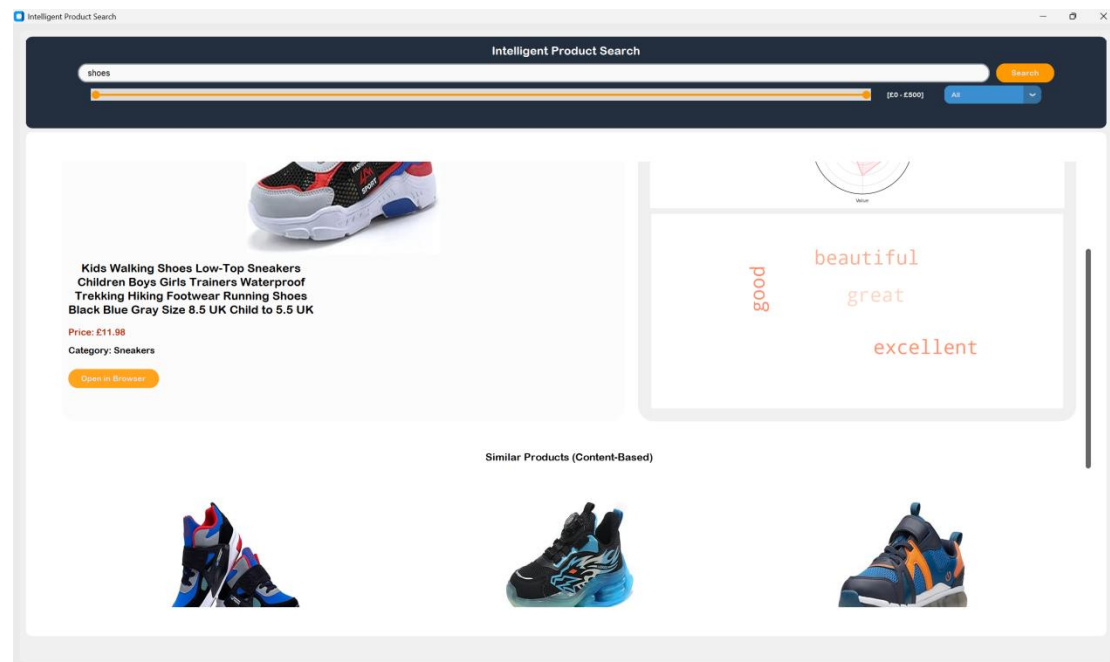


Figure 15 UI (Similarity Recommendation)

As shown in the figure below, the recommendation of user-product interaction can bring buyers a unique shopping experience. For example, if a considerable number of people in the user group who buy a certain product choose another specific category or specific product, then these products must have some direct connection with the original product. This not only plays a certain promotional role, but also helps consumers to add a whole set of products to the shopping cart conveniently, which is particularly important in some specific categories of products such as digital products, outdoor products, etc. At the same time, this recommendation is also an important punch to break the information cocoon. If you keep browsing a specific category of products, the more specific the features that buyers browse, the smaller the range of choices will be. This confinement often leads to some regrettable deviations.

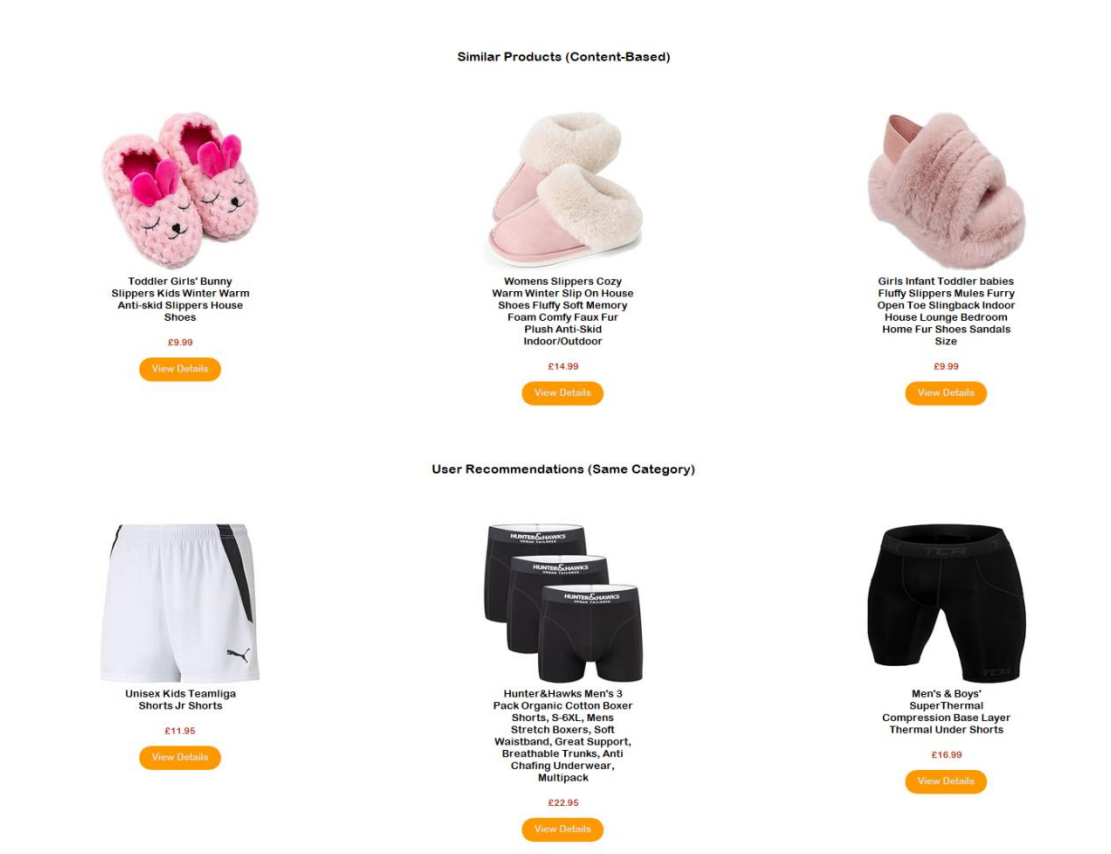


Figure 16 UI (User-Item Recommendation)

CHAPTER 7. Discussion

Judging from the visual training results, after multiple parameter adjustments and strategy upgrades, the overall accuracy of actual predictions has increased significantly. The weights of long-tail categories have also been effectively processed. Although there are many differences between different products of the same type, such as brand logos, models, styles, etc., this will inevitably interfere with the model when extracting features, resulting in a slight decrease in the overall prediction accuracy. However, for some products with small individual differences or simple appearance and clear outline features, the model still performs at a very satisfactory level, which can be intuitively reflected in the search graph and the recommendation graph of similar products above.

Unfortunately, since the processing of the original dataset requires a lot of manual annotation to ensure high enough quality for subsequent steps, we only processed the selected eleven categories. However, the accuracy of the discrimination of the Others class in the processing results also gives us sufficient confidence in the transferability of the model. As the number of manual annotation types increases, the effective scale of the dataset will become larger and larger. This does not rely on the category annotations uploaded by merchants, so it is also more scalable. Also limited by the manpower and time constraints of manually creating datasets, the application of user-item collaborative recommendation is limited to products that have already built interactive information. But judging from the performance of the model in similar and dissimilar classes, it is already sufficient for some larger-scale tasks.

CHAPTER 8. Ethic conduct

When carrying out this project, we paid enough attention to data privacy issues. Although the system needs to process product information and user data, we pay special attention to protecting user privacy. Specifically, we use a dataset publicly shared by a user named "asaniczka" on Kaggle (number DOI 10.34740/kaggle/ds/3863983). This data was collected from Amazon UK in October 2023 and has been processed in accordance with the requirements of the Open Data Commons Attribution 1.0 License. This dataset is very standardized and only contains basic product information, and does not involve personal privacy data at all. As for the user-product interaction information collected and sorted on the same website later, we also removed the time and place, de-characterized the ID, and simulated and replaced the user-product interaction data to ensure that any identifiable feature information of the original user is not leaked.

In terms of intellectual property rights, we also pay attention to compliance. We have carefully checked the use licenses of the code used in the system, such as Requests, PIL, Transformers and other open source tools. The images and text content in the dataset were also obtained legally. It should be noted that this system is purely for academic research and will never be used for commercial purposes.

Regarding data updates, the system currently uses a static data set, which means that product information and prices may not be up to date. Although technically we can update the data by crawling the Amazon website, considering that this may violate platform regulations, we decided to keep the status quo. Although doing so may affect some user experience, we believe that complying with the rules is more important.

CHAPTER 9. Conclusion

So far, we have developed a multimodal product retrieval and recommendation system that integrates both image and text information to enable relatively accurate and efficient cross-modal retrieval and similar-item recommendations. The system follows a modular design, encompassing three core modules — data preprocessing, model training and fine-tuning, and the user interface and interaction—combined into a user-friendly interface.

On the data side, the system has undergone extensive, multi-step cleaning, filtering, and reclassification of product records to ensure data quality and a reasonable distribution of categories. In the modeling aspect, the retrieval module leverages OpenCLIP and Faiss to achieve efficient text-to-image searches, while the multimodal similarity module adopts a dual-stream architecture—ResNet50 for images and BERT for text—to deeply fuse the two feature sets, facilitating accurate recommendations of similar items. The user interface, built using CustomTkinter, offers intuitive search, filtering, and recommendation features.

Although the system has shown rich and diverse functions and certain practical value, there is still room for improvement. The performance of the selected model still has some room for optimization. With more precise fine-tuning and more advanced strategies, the effect will be better than it is now. As a practical e-commerce platform, the user interface does seem a bit rudimentary; in the future, we will make improvements in layout and aesthetics to enhance the user experience.

During the implementation process, we were also amazed at how fast the technology was iterating. Bringing AI review summaries to the e-commerce platform was an independent idea from our design phase, but when we suddenly found that these imagined functions were rapidly popularized in a silent update, we were still a little overwhelmed. But this did not affect our continued progress, learning emerging technologies, and working hard to realize the simple ideas in our minds.

CHAPTER 10. Reference

- [1] Dũng C ả m L ắ ng. (2020, July 15). Paper reading: Billion-scale commodity embedding for e-commerce recommendation in Alibaba. Medium. <https://medium.com/%40dungcamlang/paper-reading-billion-scale-commodity-embedding-for-e-commerce-recommendation-in-alibaba-e97bd5d32318>
- [2] Brovman, Y. M. (2022, March 1). Building a deep learning-based retrieval system for personalized recommendations. eBay Innovation Blog. [Building a Deep Learning Based Retrieval System for Personalized Recommendations](#)
- [3] Zheng, Y., Bian, J., Meng, G., Zhang, C., Wang, H., Zhang, Z., Li, S., Zhuang, T., Liu, Q., & Zeng, X. (2022). Multi-objective personalized product retrieval in Taobao search. arXiv preprint, arXiv:2210.04170.
- [4] OpenCLIP Madasu, A., Lal, V., & Howard, P. (2025). Quantifying interpretability in CLIP models with concept consistency. arXiv preprint, arXiv:2503.11103.
- [5] Gao, P., Geng, S., Zhang, R., Ma, T., Fang, R., Zhang, Y., Li, H., & Qiao, Y. (2025). CLIP-Adapter: Better vision-language models with feature adapters. arXiv preprint, arXiv:2110.04544.
- [6] Nayak, P. (2019, October 25). Understanding searches better than ever before. The Keyword. [Understanding searches better than ever before](#)
- [7] Han, S., Wang, X., Bendersky, M., & Najork, M. (2020). Learning-to-rank with BERT in TF-Ranking. arXiv preprint, arXiv:2004.08476.
- [8] Ren, J. (2024). Multimodal sentiment analysis based on BERT and ResNet. arXiv preprint, arXiv:2412.03625. <https://doi.org/10.48550/arXiv.2412.03625>
- [9] Chen et al. (2022). "LightFormer: A Lightweight Transformer for Multimodal Recommendation" (ACM MM 2022)
- [10] Li, L. H., Yatskar, M., Yin, D., Hsieh, C.-J., & Chang, K.-W. (2019). VisualBERT: A simple and performant baseline for vision and language. arXiv preprint, arXiv:1908.03557.
- [11] Chen, Y.-C., Li, L., Yu, L., El Kholy, A., Ahmed, F., Gan, Z., Cheng, Y., & Liu, J. (2020). UNITER: UNiversal Image-TExt Representation Learning. arXiv preprint, arXiv:1909.11740.
- [12] asaniczka, "Amazon UK public reviews dataset," *Kaggle*, Oct. 2023. doi: 10.34740/kaggle/ds/3864183.
- [13] "Open Data Commons Attribution License (ODC-By) v1.0," *Open Data Commons*, 2009.
- [14] Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, Jenia Jitsev, "Reproducible scaling laws for contrastive language-image learning," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 2818-2829. Available: [2212.07143](#)
- [15] Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V. Le, Barret Zoph, Jason Wei, Adam Roberts, "The Flan Collection: Designing Data and Methods for Effective Instruction Tuning," arXiv preprint arXiv:2301.13688, 2023. Available: [2301.13688](#)

- [16] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 770-778. Available: [1512.03385](#)
- [17] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Minneapolis, MN, USA, 2019, pp. 4171-4186. Available: [1810.04805](#)
- [18] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural Collaborative Filtering," Proc. of the 26th Int. Conf. on World Wide Web (WWW), 2017, pp. 173 – 182, doi: 10.1145/3038912.3052569.
- [19] D. Cheng, M. Qi, M. Ng, and S. Liu, "Vision-and-Language Pretraining for Cross-Modal E-Commerce Retrieval," arXiv preprint arXiv:2207.00208, 2022
- [20] W. Lin, R. Li, Y. Liu, and J. Li, "Diffusion Convolutional Recurrent Neural Network: Data-driven Traffic Forecasting," *arXiv preprint* arXiv:1909.01315, 2019
- [21] OpenCLIP contributors (2022). <https://github.com/openai/CLIP/blob/main/LICENSE>
- [22] Google Research. Flan-T5: Apache License Version 2.0. GitHub Repository. <https://github.com/google-research/FLAN/blob/main/LICENSE>
- [23] K. Reitz and the Python Software Foundation, "Requests: HTTP for Humans," GitHub repository. Available: <https://github.com/psf/requests>